

AI Knowledge Compiler Architecture & Workflow

Ognjen Jovanovic

2026-04-21

AI Knowledge Compiler: Architecture & Workflow

Author: Ognjen Jovanovic

This document outlines the high-level architecture, the ingest and compilation workflows, and the synchronization strategy between components in the AI Knowledge Compiler.

1. High-Level Architecture

The AI Knowledge Compiler is built on a robust local-first architecture, leveraging **Next.js**, React, and the Vercel AI SDK to integrate multiple tools seamlessly. The core capabilities involve unstructured parsing via **grapper** and hybrid semantic search via **qmd**.

graph TD

```
A[Client UI / React] -->|API Requests| B[Next.js App Router API]
```

```
subgraph Core Services
```

```
B -->|Ingest text/URLs/PDFs| C[Extractor via grapper]
```

```
B -->|Lint & Lint Requests| D[Vercel AI SDK]
```

```
B -->|Search Queries| E[Qmd Search & Embed]
```

```
end
```

```
subgraph Data Layer
```

```
C -.->|Markdown chunks| F[(Git-Backed Vault System)]
```

```
D -.->|Analyses & Fixes| F
```

```
E -.->|Reads Models| F
```

```
end
```

```
subgraph External Dependencies
```

```
D -->|Calls| G[DeepSeek LLM APIs]
end
```

Key Components

- **Client UI:** Built with React 19, provides visual graph navigation (react-force-graph-2d) and ingest interfaces.
- **Next.js Backend:** App Router handles the API logic for scraping, semantic embedding, and search retrieval.
- **Grapper:** A CLI-based scraping utility configured locally to convert unstructured inputs (PDFs, robust web pages) into Markdown.
- **Qmd:** Local-first vector and semantic search engine that indexes the Git-backed vault, enabling lightning-fast LLM-reranked text retrieval.
- **Vault System:** A file-based, Git version-controlled knowledge graph enforcing localized state.

2. Ingestion & Linting Workflow

Transforming raw knowledge into interconnected markdown assets involves an iterative **Mint & Lint** process. The system safely buffers updates, detects inconsistencies, and provides interactive suggestions before committing.

sequenceDiagram

```
participant U as User
participant A as API
participant G as Grapper (Extractor)
participant C as Chunker
participant L as LLM (DeepSeek)
participant V as Git Vault
```

```
U->>A: Submit URL / PDF
A->>G: Extract Markdown content
G-->>A: Raw Markdown Payload
A->>C: Section-aware split
C-->>A: Markdown Sections
A->>L: Process to Nodes & Links
L-->>A: Structured JSON Updates
A->>V: Mint (Write Markdown nodes)
```

Note over U, V: Linting Phase (Asynchronous or Manual)

```
U->>A: Trigger "Mint & Lint"
A->>L: Scan Vault for orphans/contradictions
L-->>A: Issue Proposals
A->>U: Display Lint UI
```

Mint & Lint UI

Figure 1: Mint & Lint UI

U->>A: Approve Fixes
A->>V: Apply & Auto-Commit Fixes

Mint & Lint Process Snapshot

3. Synchronization: Qmd and Grapper

The interaction between the raw extractor (**grapper**) and the semantic search engine (**qmd**) ensures that newly acquired information is instantly searchable.

flowchart LR

```
Origin((External Source)) --> |PDF / Web| Grapp[Grapper CLI]
Grapp --> |Extracts| MD[Raw Markdown]
MD --> Minting[AI Node Structuring]
Minting --> Vault[(Git Vault)]
```

```
Vault --> |Triggers| QmdIndex[Qmd Embed]
QmdIndex --> |Updates| VectorDB[(Qmd Vector Index)]
```

```
Client((User Search)) --> |Query| QmdAPI[Qmd Retrieval API]
QmdAPI --> |Semantic Rerank| Client
VectorDB -.-> QmdAPI
```

1. **Extraction Pipeline:** **grapper** ingests source content (e.g., academic PDFs) with high fidelity.
2. **Structuring:** We chunk and mint these into nodes in our file system.
3. **Synchronization Trigger:** Upon a successful Git commit in the Vault (handled via async mutex to prevent conflicts), a background indexation task triggers **qmd embed**.
4. **Search Readiness:** Within seconds, the vector capabilities of **qmd** absorb the new context, allowing subsequent semantic searches to retrieve and navigate the freshly integrated concepts.

UI Screenshots Overview

Landing Page & Architecture

Knowledge Node Exploration

Landing Page

Figure 2: Landing Page

Node Overview

Figure 3: Node Overview

Generated for the AI Knowledge Compiler documentation, 2026, by Ognjen Jovanovic.